

RAK5146 / ChirpStack Concentrator / AS923 — Operator Manual

What this technology does

The RAK5146/SX1303 is Gateway-01's LoRa concentrator. ChirpStack Concentrator is the only process allowed to control that radio. It receives LoRa frames from sensors and exposes the supported event interface used independently by MQTT Forwarder and the gateway evidence journal.

Current radio contract

- Gateway: Gateway-01.
- Concentrator: RAK5146/SX1303 over SPI.
- Region: **AS923**.
- MQTT region/topic prefix: as923.
- Authoritative Gateway EUI: 0016c001f139a1cb.
- MQTT Forwarder backend: Concentrator.
- Legacy Semtech UDP Packet Forwarder: disabled.

The current project baseline uses the plain label **AS923**. Some older setup prose contains more specific historical regional wording; do not relabel the active system from documentation alone. Verify exact live frequencies/channels before any radio change.

Safety rules

1. Only one service may own the concentrator.
2. Never start the legacy UDP packet forwarder beside Concentrator.
3. Never change region/channel plan at only one layer. Sensor firmware, gateway, ChirpStack region, MQTT prefix, RX settings, and tests must remain compatible.
4. Do not raise transmit power to compensate for a receive problem.
5. Do not detach/reseat the RAK5146 while the gateway is powered.

Step 1 — Confirm Concentrator is alive

Run on Gateway-01:

```
ubus call chirpstack-concentratord-sx1302 concentratord_stats
service chirpstack-concentratord-sx1302 status
```

PASS means: the service answers without an IPC/UBus error and returns live concentrator statistics.

If the service name differs after an intentional package change, discover it first rather than guessing:

```
ps w | grep '[c]hirpstack-concentratord'
uci show chirpstack-concentratord
```

Step 2 — Confirm hardware identity from the active startup

```
logread -e chirpstack-concentratord | tail -n 120
```

Look for the successful SX1302/SX1303 startup and the retrieved Gateway ID.

PASS means: the active process reports Gateway EUI 0016c001f139a1cb and does not repeatedly fail SPI reset/initialization.

Do not: select an arbitrary 16-hex value from inactive SX1301 configuration. The running RAK5146 startup is authoritative.

Step 3 — Confirm effective radio configuration

```
uci show chirpstack-concentratord
```

Verify the active configuration indicates:

- enabled service;
- SX1302/SX1303 chipset family;
- RAK5146 model/profile;
- AS923 region/channel plan;
- SPI rather than an unrelated USB concentrator mode.

If the live configuration exposes exact channel frequencies, record those before changing anything. Do not infer them from a generic AS923 table when diagnosing a live mismatch.

Step 4 — Confirm no competing packet forwarder is active

```
uci show chirpstack-udp-forwarder
ps w | grep -E '[u]dp-forwarder|[p]acket_forwarder' || true
```

PASS means: there is no enabled remote UDP packet-forwarder path competing with Concentrator/MQTT Forwarder.

Step 5 — Confirm the local RF-to-MQTT handoff

First confirm local Mosquitto is listening:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

It must be loopback-only. Then, in one shell, subscribe to the gateway event hierarchy:

```
mosquitto_sub -h 127.0.0.1 -p 1883 -t 'as923/gateway/0016c001f139a1cb/event/#' -v
```

Generate one approved real sensor uplink.

PASS means: a message arrives on the correct EUI/topic. The payload may appear binary because the gateway path uses Protobuf.

If no message arrives: stop here. Do not edit cloud certificates. Check sensor transmission, AS923 compatibility, Concentrator event reception, MQTT Forwarder backend, and local Mosquitto.

Step 6 — Distinguish common OTAA failures

If JoinRequest is visible but JoinAccept never completes, inspect in this order:

1. sensor and gateway regional/channel compatibility;
2. ChirpStack `region_as923` configuration;
3. DevEUI and JoinEUI registration;
4. AppKey/NwkKey material in the correct protected source;
5. RX1/RX2 parameters and device profile;
6. downlink scheduling/radio transmit evidence.

Do not repeatedly rotate keys or re-register the device until RF/channel agreement has been proven.

Step 7 — Verify evidence observation after a real uplink

The gateway journal must consume the supported Concentrator event interface independently of MQTT Forwarder. Use the evidence-service manual to confirm one new journal sequence corresponds to the same known test uplink.

A normal application event with no source evidence is **not** a complete v2 evidence result.

Change procedure

Before changing radio configuration:

1. save an off-gateway Gateway OS backup;
2. record current `uci show chirpstack-concentrator` output without secrets;
3. record the current Gateway EUI and channel/frequency values;
4. change only the intended radio setting;
5. Save & Apply once;
6. wait for SPI initialization to finish;
7. rerun Steps 1–5;
8. if a regional parameter changed intentionally, verify sensor firmware and ChirpStack before generating traffic.

Do not repeatedly click **Save & Apply** while the radio is initializing.

Troubleshooting order

Symptom	Likely layer
SPI/reset errors, no Gateway EUI	HAT seating, SPI/GPIO profile, power, Concentratord
Stable EUI but zero RF events	antenna, sensor TX, frequency/channel/SF/BW mismatch
Local RF/MQTT events but no cloud	local broker bridge/LTE/TLS
JoinRequest received but no JoinAccept	keys/profile/RX/downlink/region mismatch
App data present but evidence missing	evidence journal/interface, not radio delivery

Continuous GPS/GNSS warnings are not proof that RF is broken. Diagnose GPS separately unless the selected test actually requires GNSS timing.

Recovery

For a service-only failure, repair configuration/package/service state before considering a reflash. For a suspected hardware failure, power down before reseating or replacing the concentrator. After replacement, do not assume the Gateway EUI is unchanged; retrieve it from the active RAK5146 startup and deliberately update every identity-bound dependency if the EUI changed.

Completion checklist

- Concentratord stats respond.
- service is not crash-looping.
- active Gateway EUI is 0016c001f139a1cb unless hardware was intentionally replaced.
- current region is AS923.
- no competing UDP forwarder is active.
- one real uplink reaches the local as923/gateway/.../event/# topic.
- the evidence path can observe the source event independently.

Detailed runbooks

- [../gateway/setup/01-hardware-assembly.md](#)
- [../gateway/setup/03-configure-concentratord.md](#)
- [../gateway/setup/05-configure-mqtt-forwarder.md](#)
- [../gateway/setup/06-verify-gateway-os.md](#)
- [../gateway/operations/06-rf-planning-and-site-survey.md](#)

Supporting runbook: 01-hardware-assembly.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 1. Assemble the Raspberry Pi 4B and RAK5146 SPI

This procedure installs the RAK5146 SPI concentrator on its Raspberry Pi HAT and mounts the HAT on a Raspberry Pi 4B.

Keep every power source disconnected until the antenna and all boards are installed.

What you need

- Raspberry Pi 4B
- RAK5146 **SPI** concentrator
- compatible RAK Pi HAT for the RAK5146
- mounting screws and standoffs supplied with the HAT
- LoRa antenna matched to the concentrator frequency band
- u.FL-to-SMA pigtail when the HAT or enclosure uses an external SMA connector
- stable Raspberry Pi USB-C power supply
- microSD card for ChirpStack Gateway OS Base

The RAK5146 uses an mPCIe-shaped connector, but the module in this manual communicates through SPI. Do not install it in a laptop or normal PCIe slot.

Step 1: Check the module and antenna labels

Read the labels before assembling anything.

Confirm:

```
Raspberry Pi model: Raspberry Pi 4B
Concentrator model: RAK5146-115 (Onboard GPS, Non-LBT)
Concentrator interface: SPI
Concentrator frequency band: 915 MHz High-Frequency (902-928 MHz)
Target region: Philippines (AS923 / AS923-1, 920-925 MHz)
Antenna frequency band: 900-930 MHz (Matches concentrator & AS923 spectrum)
```

The software channel plan can only use frequencies supported by the installed radio module and antenna. Stop here when the module label, interface, or antenna band is uncertain.

Step 2: Fit the RAK5146 into the Pi HAT

1. Place the Pi HAT on a clean, non-conductive surface.
2. Hold the RAK5146 by its edges.
3. Align the module edge connector with the HAT socket.
4. Insert the module at a shallow angle until the connector is fully seated.
5. Press the free end of the module down onto the mounting posts.
6. Install the retaining screws and tighten them only until the module is secure.

The module should sit flat and parallel with the HAT. Remove and reseal it when the gold contacts remain visibly exposed or one side is higher than the other.

Step 3: Connect the LoRa pigtail

The small u.FL connector is easy to damage. Press it straight down; do not slide or twist it into place.

1. Find the RAK5146 connector labelled for the LoRa RF path.
2. Hold the pigtail connector directly above it.
3. Check that both connectors are centered.
4. Press straight down until the connector snaps into place.
5. Route the cable without sharp bends, pinching, or tension.
6. Secure the external SMA end to the enclosure or HAT bracket when one is used.

Do not connect the LoRa pigtail to a GNSS connector. Use the board labels rather than the connector position alone.

Step 4: Mount the HAT on the Raspberry Pi

1. Install the Raspberry Pi standoffs in the mounting holes.
2. Check the 40-pin Raspberry Pi header for bent pins.
3. Align the HAT socket with all 40 pins. Verify that it is not shifted by one row or one column.
4. Press evenly above the header until the HAT is fully seated.
5. Fasten the HAT to the standoffs.
6. Check the underside of the HAT for contact with the Raspberry Pi USB, Ethernet, or display connector shells.

Do not force the HAT onto a misaligned header. A shifted connector can place supply voltage on the wrong signal pins.

Step 5: Attach the LoRa antenna

1. Confirm again that the antenna band matches the RAK5146 frequency variant.
2. Screw the antenna or antenna feed cable onto the LoRa SMA connector.
3. Tighten it by hand until secure.
4. Position the antenna vertically for normal bench testing.

Always attach a suitable antenna before powering the gateway or enabling the concentrator transmitter. Transmitting into an open RF connector can damage the radio front end.

Step 6: Perform the final visual check

Before inserting the microSD card or connecting power, verify:

- the RAK5146 is the SPI model;
- the module is fully seated and secured;
- the u.FL connector is centered and snapped into place;
- the pigtail is not trapped under a board or screw;
- the HAT uses all 40 header pins without an offset;
- the HAT is supported by standoffs;
- no loose screw or metal part is under either board;
- the LoRa antenna is connected to the correct SMA port.

Common assembly problems

The RAK5146 will not lie flat

Remove the screws, lift the module, and insert the edge connector farther into the socket. Do not pull the module flat with the screws.

The u.FL connector will not snap into place

Lift it away and realign it. Pressing harder while it is off-center can damage both connectors.

The HAT does not align with the mounting holes

Check that the 40-pin header is not shifted. Also confirm that the standoff height matches the supplied HAT hardware.

The Raspberry Pi power LED is unstable after power is connected

Disconnect power immediately. Inspect for a shifted HAT, a loose metal object, or an unsuitable power supply before trying again.

Next step

Leave the antenna attached and continue with [02-install-chirpstack-gateway-os.md](#).

Supporting runbook: 03-configure-concentrator.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 3. Configure Concentrator for RAK5146-115 (Philippines AS923)

This procedure configures **Concentrator** (the ChirpStack radio control daemon) to communicate with the **RAK5146-115** SPI LoRaWAN concentrator board on the Raspberry Pi 4B, specifically tailored for the **Philippines (AS923 / AS923-1)** LoRaWAN region.

Concentrator interfaces directly with the SX1303 baseband processor over the SPI bus. **MQTT Forwarder** consumes the supported Concentrator event interface for the delivery path. The software integrity journal added in Gateway 4A consumes that same supported event stream independently for evidence. Neither downstream service may take ownership of the SPI radio.

[!CAUTION] Only one service may control the LoRa concentrator hardware at a time. Never run another packet forwarder daemon (such as the legacy Semtech UDP packet forwarder) simultaneously on the same radio hardware.

Hardware and Regional Specifications

Confirm these confirmed hardware and regional parameters before enabling radio transmissions:

```
Module model: RAK5146-115
Bus interface: SPI
Hardware frequency variant: 915 MHz High-Frequency (902-928 MHz)
Onboard features: Integrated ZOE-M8Q GPS / Non-LBT
Baseband chipset: Semtech SX1303
Deployment country: Philippines
Approved LoRaWAN region: AS923 (AS923-1 / Group 1)
Frequency spectrum: 920 MHz - 925 MHz (Join channels: 923.2 MHz & 923.4 MHz)
Selected channel plan: AS923 (or AS923_1)
MQTT region topic prefix: as923
Antenna frequency band: 900-930 MHz (Matches RAK5146-115)
```

[!IMPORTANT] The RAK5146-115 uses the 915 MHz high-frequency front-end (902-928 MHz), which fully covers the Philippines AS923-1 frequency band (920-925 MHz). Selecting AS923 in Gateway OS configures the correct center frequencies and multi-channel decoders for the Philippines.

Step 1: Open the Concentrator page

What this step does

Navigates to the Concentrator management interface within OpenWrt LuCI (**ChirpStack > Concentrator**).

Why we do it

Opens the graphical configuration interface where Gateway OS configures baseband hardware drivers, SPI device paths (`/dev/spidev0.0`), GPIO reset pin mappings, and frequency channel plans for the concentrator HAT.

Procedure

1. Sign in to the Gateway OS LuCI web interface (`http://<GATEWAY_IP>/`).
2. Navigate to **ChirpStack > Concentrator**.
3. Locate the configuration section for the concentrator card.

Step 2: Select the RAK5146 profile and AS923 channel plan

What this step does

Enables Concentrator and configures it to use the RAK5146 hardware driver profile matched with the **AS923** regional channel plan for the Philippines.

Why we do it

- **Hardware Reset & SPI Mapping:** The RAK5146-115 uses an SX1303 baseband processor connected over SPI. Selecting the dedicated RAK5146 profile ensures Gateway OS uses the correct GPIO pin numbers to reset the board and initializes the correct SPI bus interface (`/dev/spidev0.0`).
- **Frequency & Channel Decoding:** Selecting AS923 configures center frequencies for Radio 0 (923.2 MHz) and Radio 1 (924.0 MHz) and binds multi-SF multi-channel decoders so the gateway can receive and decode node transmissions across the 920–925 MHz spectrum used in the Philippines.

Procedure

Tab 1: Global configuration

1. In LuCI, open **ChirpStack > Concentrator**.
2. Stay on the **Global configuration** tab.
3. Configure: * **Enabled:** Checked [x] * **Enabled chipset:** SX1302 / SX1303

Tab 2: SX1302 / SX1303 configuration

1. Select the **SX1302 / SX1303** tab at the top of the page.
2. Configure the hardware fields:

Field Name	Setting Value	Purpose / Requirement
Antenna gain (dBi)	2 (or 3 depending on antenna)	Matches your physical omni antenna.
Shield model	RAK - RAK5146	Binds SPI GPIO reset and SX1303 driver.

Field Name	Setting Value	Purpose / Requirement
Channel-plan	AS923 - Standard channels + 923.6, 923.8, ... 924.6	Philippines AS923-1 channel allocation.
Gateway ID (optional)	(Leave empty)	Auto-detects factory EUI from SX1303 chip.
GNSS	Checked [x]	Enables uBlox ZOE-M8Q GPS on RAK5146-115.
USB	Unchecked []	RAK5146-115 uses SPI bus, not USB.

[!IMPORTANT] * **GNSS Checkbox:** Must be **Checked [x]** because your RAK5146-115 module includes an onboard uBlox GNSS module for fine time-stamping. * **USB Checkbox:** Must be **Unchecked []** because your module communicates over the SPI bus interface. * **Gateway ID Field and inactive SX1301 values:** * **Leave Gateway ID (optional) EMPTY on the active sx1302 / sx1303 tab.** * When the global chipset is sx1302, ignore any gateway_id displayed under inactive SX1301 configuration. * The stale SX1301 example 2ccf67fffe0abee3 is not the RAK5146 identity and must never be registered in ChirpStack. * The authoritative EUI is the Gateway ID retrieved value Concentratord reports after the active RAK5146 successfully starts.

3. Select **Save & Apply** at the bottom right.

Step 3: Review antenna gain and transmit power settings

What this step does

Configures antenna gain (dBi) and cable loss parameters matching your physical 900–930 MHz antenna installation.

Why we do it

- **Regulatory Compliance (EIRP Limits):** Transmit power in the Philippines must adhere to NTC (National Telecommunications Commission) limits for short-range wireless devices. Entering accurate antenna gain allows Concentratord to calculate proper power attenuation for downlink transmissions.
- **Downlink Reliability:** Setting inaccurate gain values can cause downlink Join-Accept packets or Class A/C acknowledgments to fail or be transmitted at incorrect power levels.

Procedure

When antenna gain, cable loss, LBT (Listen Before Talk), or transmit power fields are displayed:

1. Enter the verified gain for your 900–930 MHz antenna (typically 2 or 3 dBi for standard omni antennas).
2. Keep regional default values when no site-specific calibration data exists.
3. Do not arbitrarily increase transmit power to solve receive sensitivity issues.
4. Do not copy antenna settings from a different region (e.g. EU868 or US915).

Step 4: Save and start Concentratord

What this step does

Saves the configuration to OpenWrt UCI storage (`uci commit`) and starts the `chirpstack-concentratord` background daemon via `Monit` / `systemd`.

Why we do it

Applies power to the RAK5146-115 HAT, executes the GPIO reset pulse sequence, loads radio firmware into the SX1303 baseband chip over SPI, and starts real-time multi-channel spectrum listening on AS923 frequencies.

Procedure

1. Select **Save & Apply**.
2. Wait for LuCI to finish applying the configuration.
3. Allow 10–15 seconds for the SPI hardware initialization sequence to complete.
4. Refresh the browser page.

[!CAUTION] Do not repeatedly click **Save & Apply** while the service is initializing over SPI.

Step 5: Confirm and record the Gateway EUI

What this step does

Reads and displays the unique 64-bit (16 hexadecimal character) **Gateway EUI** generated from the RAK5146-115 concentrator chip's internal hardware ID.

Why we do it

The **Gateway EUI** is the permanent hardware identity of your gateway. It is required for: * Registering the gateway on the ChirpStack Network Server. * Generating the gateway's MQTT topic hierarchy (`as923/gateway/<GATEWAY_EUI>/...`). * Provisioning TLS client certificates and setting broker Access Control List (ACL) permissions.

Procedure

After clicking **Save & Apply**, `Concentratord` initializes the RAK5146 hardware over SPI and extracts the chip's unique 16-character factory Gateway EUI.

Method A: Read the authoritative EUI from Concentratord startup logs

1. SSH into the gateway from your administration workstation terminal: `bash ssh root@<GATEWAY_IP>`
2. Read the successful `Concentratord` startup directly: `sh logread -e chirpstack-concentratord | tail -n 100`
3. Filter hardware/identity lines when needed: `sh logread -e chirpstack-concentratord | \ grep -Ei 'gateway|eui|sx130|rak5146|error|fail'`
4. Record the EUI from: text `Gateway ID retrieved, gateway_id: "<REAL_GATEWAY_EUI>"`

Do not scrape an arbitrary 16-hex string from all configuration/log text. The running active concentrator is the authority.

Method B: Check LuCI Status Page

1. Refresh the Concentrator page (**ChirpStack > Concentrator**) or open **Status > Overview**.
2. Locate the **Gateway ID** or **Gateway EUI** display field.

Record the Gateway EUI

Record the 16-character hexadecimal string as <GATEWAY_EUI> (e.g. ac1f09fffe05abcd).

[!IMPORTANT] The Gateway EUI is permanent and tied to the RAK5146 hardware. Reboot the gateway once and verify <GATEWAY_EUI> remains identical across reboots.

Step 6: Verify the effective configuration over SSH

What this step does

Connects to the gateway via SSH and inspects saved OpenWrt Unified Configuration Interface (UCI) options and background daemon status.

Why we do it

Confirms that configuration values persisted correctly in file storage (/etc/config/chirpstack-concentrator) and that Monit reports the service is running healthily without crash-reboot loops.

Procedure

Run these commands in an SSH terminal:

```
uci show chirpstack-concentrator
/etc/init.d/chirpstack-concentrator status
ps w | grep concentrator
```

Verify that: * chirpstack-concentrator.@global[0].enabled='1' * chirpstack-concentrator.@global[0].chipset='sx1302' * chirpstack-concentrator.@sx1302[0].model='rak_5146' (or rak5146) * chirpstack-concentrator.@sx1302[0].channel_plan='as923' * chirpstack-concentrator.@sx1302[0].gnss='1' * Service status reports **running** and ps w shows active chirpstack-concentrator processes.

Step 7: Troubleshoot hardware detection or a missing Gateway EUI

What this step does

Inspects OpenWrt system logs (logread) to diagnose SPI communication failures, pin alignment errors, or missing Gateway EUI issues.

Why we do it

Identifies physical layer issues (unseated HAT, wrong 40-pin GPIO alignment, disabled SPI interface, or wrong voltage rail) before proceeding to software network configuration.

Procedure

If the Gateway EUI is missing or Concentratord fails to start, view the log output over SSH:

```
logread -e chirpstack-concentratord
```

Match the error output against this guide:

Log Message / Error	Root Cause & Action
SPI device cannot be opened	Check RAK5146 SPI variant, 40-pin GPIO header alignment, and Raspberry Pi SPI interface status.
Reset or GPIO error	Check HAT seating, pin alignment, and confirm RAK5146 profile is selected.
SX1250 or calibration error	Check module frequency variant (confirm RAK5146-115 915MHz model), power supply, and antenna connection.
Service repeatedly restarts	Inspect UCI saved settings (<code>uci show chirpstack-concentratord</code>) for invalid channel plan names.
Gateway EUI changes after reboot	Check for competing packet forwarder services or wrong shield profile selection.

[!CAUTION] Always power down the Raspberry Pi before reseating the 40-pin HAT or RAK5146 module.

Step 8: Disable legacy UDP Packet Forwarder

What this step does

Disables and stops the legacy Semtech UDP packet forwarder service (`chirpstack-udp-forwarder`) in OpenWrt.

Why we do it

- Prevent Hardware Conflicts:** Multiple daemons attempting to control the same SPI bus interface simultaneously will cause bus collision errors and crash the radio.
- Prevent Packet Duplication & Insecure Transmission:** Legacy UDP forwarders transmit unencrypted packet data over UDP port 1700. Disabling UDP Forwarder ensures all radio traffic flows exclusively through the secure Concentratord -> MQTT Forwarder architecture.

Procedure

- In LuCI, navigate to **ChirpStack > UDP Forwarder**.
- On the **Global configuration** tab, ensure **Enabled** is **Unchecked** [].
- Select **Save & Apply**.
- Verify over SSH that UDP Forwarder remains disabled:

```
uci show chirpstack-udp-forwarder
```

Expected SSH Output:

```
chirpstack-udp-forwarder.@global[0].enabled='0'
```

Step 9: Confirm the MQTT region topic prefix (as923)

What this step does

Inspects the MQTT topic prefix generated by ChirpStack Gateway OS for the **AS923** regional channel plan.

Why we do it

MQTT Forwarder constructs MQTT payload topics using the format `<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/<EVENT_TYPE>`. For this Philippines AS923 / AS923-1 setup, the working MQTT topic prefix is exactly `as923`. Keep that same value in MQTT Forwarder, the local bridge, server ACLs, and ChirpStack region backend.

Procedure

Inspect the topic prefix in LuCI under **ChirpStack > MQTT Forwarder**, or read it over SSH:

```
uci show chirpstack-mqtt-forwarder | grep -i topic
```

Confirm the output value is:

```
as923
```

Record this value as `<REGION_TOPIC_PREFIX>`.

Step 10: Prepare the gateway MQTT identity

What this step does

Links the confirmed `<GATEWAY_EUI>` to the downstream server provisioning workflow to prepare TLS certificates.

Why we do it

Secure gateway communication uses Mutual TLS (mTLS). The remote ChirpStack Network Server generates client certificates where the **Common Name (CN)** strictly matches your `<GATEWAY_EUI>`.

Procedure

Complete the server-side identity provisioning manual:

[Provision the gateway MQTT identity](#)

Obtain the following files for deployment in manual 04: * Remote MQTT Broker CA Certificate (`ca.crt`) * Gateway Client Certificate (`<GATEWAY_EUI>.crt`) * Gateway Private Key (`<GATEWAY_EUI>.key`) * Remote Broker FQDN / Hostname and TLS Port

Completion check

Before moving to the next setup guide, verify that: - [] `chirpstack-concentratord` is active and running (`monit status`). - [] RAK5146-115 SPI profile is selected. - [] Philippines **AS923** channel plan is active. - [] `<GATEWAY_EUI>` is recorded and remains stable across reboots. - [] `<REGION_TOPIC_PREFIX>` is confirmed as `as923`. - [] `chirpstack-udp-forwarder` is completely disabled.

Next step

Continue with [04-configure-local-mqtt-buffer.md](#), then add the independent [04a-configure-gateway-integrity-journal.md](#) before final gateway verification.

Supporting runbook: 05-configure-mqtt-forwarder.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 5. Configure ChirpStack MQTT Forwarder

MQTT Forwarder receives packets from Concentrator and publishes ChirpStack gateway messages to the local Mosquitto broker. It is the **delivery path**, not the gateway evidence authority.

The independently reviewed journal from Gateway 4A consumes the supported Concentrator event interface beside MQTT Forwarder. This separation lets the server later compare what the journal recorded with what MQTT actually delivered.

For this setup, MQTT Forwarder must not connect directly to the remote broker. The local broker provides TLS, topic bridging, and persistent uplink buffering.

Complete [04-configure-local-mqtt-buffer.md](#) and [04a-configure-gateway-integrity-journal.md](#) before final acceptance.

Before you begin

Have these values ready:

```
Gateway EUI: <GATEWAY_EUI>
Region topic prefix: <REGION_TOPIC_PREFIX>
Local broker address: tcp://127.0.0.1:1883
Forwarder client ID: gw-fwd-<GATEWAY_EUI>
```

Confirm the local broker is running:

```
ps w | grep '[m]osquitto'
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Continue only when port 1883 is bound to 127.0.0.1.

Step 1: Open the MQTT Forwarder page

1. Sign in to the Gateway OS web interface.
2. Open **ChirpStack > MQTT Forwarder**.
3. Select the forwarder instance connected to the configured RAK5146.

The exact field names can vary between Gateway OS releases. Match the settings by purpose rather than copying an option name from another release.

Step 2: Enable the Concentratord backend

Set the backend fields to the equivalent of:

```
Enabled: yes
Backend: concentratord
```

Do not select `semtech_udp`. Concentratord is already controlling the RAK5146 and provides the packet stream used by MQTT Forwarder.

Step 3: Configure packet filtering

Use these normal forwarding settings:

```
Forward CRC OK packets: enabled
Forward CRC invalid packets: disabled
Forward packets without CRC: disabled unless required by the confirmed radio path
```

CRC-invalid packets are useful only for controlled RF diagnostics. Forwarding them during normal operation adds unusable traffic and can hide radio problems.

When the installed release provides a filter for non-LoRaWAN payloads, keep the setting consistent with the intended network. A normal LoRaWAN-only gateway can filter proprietary non-LoRaWAN frames.

Step 4: Configure the local MQTT connection

Set:

```
Server: tcp://127.0.0.1:1883
Topic prefix: <REGION_TOPIC_PREFIX>
QoS: 1
Clean session: false
Client ID: gw-fwd-<GATEWAY_EUI>
Use JSON: disabled
Username: empty
Password: empty
CA certificate: empty
Client certificate: empty
Private key: empty
```

Why these values are used

- 127.0.0.1 keeps the first MQTT hop inside the gateway.
- QoS 1 makes the local broker acknowledge gateway event and state messages.
- A fixed client ID prevents every restart from appearing as a different client.
- A non-clean session supports reliable reconnect behavior between MQTT Forwarder and the local broker.
- JSON is disabled because ChirpStack uses the Protobuf gateway message format on this path.
- TLS fields are empty because the loopback connection does not leave the gateway. The local Mosquitto bridge holds the remote TLS files.

Do not enter `<MQTT_BROKER_FQDN>` on this page. Doing so bypasses the persistent local queue and can also create unexplained disagreement with the independently designed journal/reconciliation path.

Step 5: Review metadata and command topics

When the page exposes event, command, state, or metadata topic templates, keep the standard ChirpStack gateway structure under the exact region prefix:

```
<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/...  
<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/state/...  
<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/command/...
```

Do not remove the Gateway EUI level or replace it with a hostname. The remote broker ACL and ChirpStack gateway lookup depend on this topic structure.

Step 6: Save the configuration

1. Recheck the Gateway EUI and region prefix.
2. Select **Save & Apply**.
3. Wait for MQTT Forwarder to restart.
4. Refresh the page and confirm the saved values remain present.

Step 7: Inspect the effective UCI configuration

Run over SSH:

```
uci show chirpstack-mqtt-forwarder  
monit status
```

Confirm that the effective configuration shows:

- the Concentrator backend;
- tcp://127.0.0.1:1883;
- <REGION_TOPIC_PREFIX>;
- QoS 1;
- the fixed gw-fwd-<GATEWAY_EUI> client ID;
- JSON disabled;
- no remote CA, certificate, or key path.

If the LuCI page does not expose QoS, clean-session, or client-ID fields, inspect the installed package before changing anything:

```
opkg list-installed | grep 'chirpstack-mqtt-forwarder'  
ps w | grep '[c]hirpstack-mqtt-forwarder'  
uci show chirpstack-mqtt-forwarder
```

Use only options supported by that package and its generated configuration. Do not guess UCI keys or edit the init script.

Step 8: Verify a local gateway event

Open one SSH session and subscribe to this gateway's event topics:

```
mosquitto_sub -h 127.0.0.1 -p 1883 \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/#' -v
```

Generate one real uplink from an approved LoRaWAN test device.

The subscription should display a topic for the configured Gateway EUI. The payload is binary Protobuf and may appear unreadable in the terminal; the important check here is that a message arrives on the correct topic.

Press **Ctrl+C** after the test.

No message means the failure is before the remote bridge. Check Concentratord, the region prefix, the Gateway EUI, and the local broker before changing remote certificates.

Step 9: Verify the local listener remains private

Run:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

The listener must remain 127.0.0.1:1883. The empty username and password fields are safe only for this loopback-only listener.

Step 10: Verify UDP Forwarder remains disabled

Run:

```
uci show chirpstack-udp-forwarder
```

Remove any enabled remote UDP server through **ChirpStack > UDP Forwarder**. Do not keep UDP as a second path or fallback.

Troubleshooting

MQTT Forwarder reports connection refused

Check Mosquitto first:

```
ps w | grep '[m]osquitto'
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Start or repair the local broker. Do not replace 127.0.0.1 with the remote broker address.

No local event appears after an uplink

Check the path in order:

1. Concentratord has a stable Gateway EUI.
2. The device and gateway use the same region and channel plan.
3. MQTT Forwarder uses the Concentratord backend.
4. The topic prefix matches the active region.
5. The local Mosquitto listener is running.

Use the forwarder service messages only after checking the saved settings:

```
logread -e chirpstack-mqtt-forwarder
```

The topic uses the wrong region prefix

Return to Concentratord, confirm the exact channel plan, then update MQTT Forwarder and the local Mosquitto bridge to the same <REGION_TOPIC_PREFIX>. The remote ChirpStack server must enable that same region.

MQTT Forwarder repeatedly restarts

Compare the UCI values with the installed package's supported fields. Remove unsupported guessed options and reapply the configuration through LuCI.

Local events work but ChirpStack sees nothing

MQTT Forwarder and Concentratord are working. Continue troubleshooting at the local Mosquitto bridge, remote certificate, remote ACL, broker subscription, and ChirpStack region.

Completion check

Before continuing:

- MQTT Forwarder uses the Concentratord backend;
- the server is `tcp://127.0.0.1:1883`;
- QoS 1 is enabled;
- the client ID is fixed and gateway-specific;
- the topic prefix matches the radio and ChirpStack region;
- JSON is disabled;
- a real local event appears under the correct Gateway EUI;
- no remote TLS files are configured in MQTT Forwarder;
- UDP Forwarder is disabled;
- the evidence journal remains a separate read-only Concentratord consumer and MQTT Forwarder has no permission to rewrite its files.

Next step

Continue with [06-verify-gateway-os.md](#).

Supporting runbook: 06-verify-gateway-os.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 6. Verify Gateway OS, MQTT Buffering, and Integrity

This procedure verifies both gateway paths one layer at a time:

```
delivery -> MQTT Forwarder -> Mosquitto -> remote MQTT
evidence -> integrity journal -> hash chain -> cloud checkpoint/segment upload
```

Neither path is considered proven merely because its process is running.

A service showing as running is not enough. The checks must prove that:

- the RAK5146 is using the correct radio configuration;
- MQTT Forwarder publishes to the local broker;
- the local broker reaches the remote broker with the gateway certificate;
- uplinks remain queued during a WAN outage;
- the queue survives a gateway reboot;
- queued uplinks drain after connectivity returns;
- old downlink commands are not replayed after the outage;
- the journal records the same real uplink source independently of MQTT Forwarder;
- sequence and record/segment hashes remain continuous across normal operation and reboot;
- a WAN outage does not stop local journaling or create a false server checkpoint;
- recovery uploads missing segments and extends the last accepted off-device checkpoint;
- the server can reconcile journal evidence with the delivered remote gateway MQTT event.

Run the outage and reboot tests in staging or during an approved maintenance window.

Before you begin

Have these values ready:

```
Gateway address: <GATEWAY_IP>
Gateway EUI: <GATEWAY_EUI>
Region topic prefix: <REGION_TOPIC_PREFIX>
Remote broker: <MQTT_BROKER_FQDN>:8883
Expected queue free-space reserve: <BUFFER_FREE_SPACE_RESERVE_BYTES>
Journal implementation/version: <PINNED_JOURNAL_VERSION>
Journal storage budget: <JOURNAL_STORAGE_BUDGET>
Evidence ingest endpoint: https://<EVIDENCE_INGEST_FQDN>
```

For the real radio tests, use an approved OTAA device configured for the same region and channel plan as the gateway and ChirpStack server.

When the gateway or device has not yet been registered in ChirpStack, complete the relevant sections of [Register and test the gateway](#), then return to the traffic tests in this manual.

Step 1: Verify Gateway OS and network access

Run over SSH:

```
cat /etc/os-release
uname -a
date -u
ip addr
ip route
nslookup <MQTT_BROKER_FQDN>
monit status
ps w | grep '[m]osquitto'
```

Confirm that:

- the expected ChirpStack Gateway OS Base release is running;
- the UTC date and time are correct;
- the intended backhaul interface has an address;
- a default route exists;
- the remote broker name resolves to the expected address;
- Concentrator and MQTT Forwarder remain running;
- the local Mosquitto process is running;
- when the reviewed journal implementation has been installed, its service and uploader are healthy without repeatedly restarting.

Fix network or time problems before testing TLS. An incorrect clock can make a valid certificate appear expired or not yet valid.

For the commissioned Gateway-01 field topology, the normal production backhaul is the SIM7600 QMI interface `wwan0`; RJ45/`br-lan` at `192.168.20.11` is management-only and must not become the Internet default. After boot, allow the modem and one `lte-route-health` cycle to converge, then verify that the service is running, QMI has a current address/gateway, the metric-10 default is on `wwan0`, the production broker route resolves through `wwan0`, and UTC has synchronized. The controller intentionally retains a valid LTE default during a temporary broker/TLS failure so DNS/NTP and autonomous recovery remain possible.

```
ubus call network.interface.lte status
ubus call service list '{"name":"lte-route-health"}'
ip route show default
ip route get 129.212.208.168
date -u
logread | grep 'lte-route-health' | tail -n 20
```

On the 2026-09-15 reboot acceptance, DITO assigned a fresh PDP address, `lte-route-health` restored the metric-10 `wwan0` default automatically on its next cycle, the production MQTT mTLS probe passed, NTP corrected the clock, and the formal research-recorder preflight subsequently exited `0`. Treat a very early post-boot snapshot taken before the first successful health cycle as a startup observation, not as a routing failure; the route must appear automatically within the bounded controller cycle without a manual route command. The 2026-09-16 controller revision also distinguishes broker failure from LTE dataplane failure and performs staged autonomous recovery: bounded logical-interface restart first, then a cooldown-protected SIM7600 AT+CFUN=1,1 reset only if the

dataplane remains unavailable. Both stages were live-proven before the reboot-persistence check. The reboot-persistence check also passed: the daemon started from `S951te-route-health`, retained the repository script hash, used the 60-second boot grace instead of resetting the modem during QMI startup, then restored a fresh DITO PDP session, metric-10 default, and production mTLS without operator intervention.

Step 2: Verify the effective gateway configuration

Run:

```
logread -e chirpstack-concentratord | tail -n 100
uci show chirpstack-concentratord
uci show chirpstack-mqtt-forwarder
uci show chirpstack-udp-forwarder
```

Check the saved values against the setup:

- Concentratord uses the RAK5146 SX1302/SX1303 profile.
- The configured channel plan is the exact approved regional variant.
- The successful active SX1302/SX1303 startup log reports the expected 16-hexadecimal Gateway EUI; ignore inactive SX1301 gateway_id values.
- MQTT Forwarder uses the Concentratord backend.
- MQTT Forwarder connects to `tcp://127.0.0.1:1883`.
- MQTT Forwarder uses QoS 1 and the fixed `gw-fwd-<GATEWAY_EUI>` client ID when supported by the installed release.
- The MQTT topic prefix is `<REGION_TOPIC_PREFIX>`.
- UDP Forwarder has no enabled remote server.

Inspect both active broker files without displaying the private key:

```
grep -E
'^ (persistence|persistence_location|persistence_file|autosave_interval|max_queued_messages|max_queued_bytes|l
istener|protocol|allow_anonymous|include_dir)' \
/etc/mosquitto/mosquitto.conf

grep -E '^ (connection |address
|remote_clientid|cleansession|bridge_cacfile|bridge_certfile|bridge_keyfile|bridge_insecure|topic )' \
/etc/mosquitto/conf.d/bridge.conf
```

Confirm that:

- `persistence` is enabled;
- message and byte limits are finite;
- the listener is `1883 127.0.0.1`;
- both bridge addresses use `<MQTT_BROKER_FQDN>:8883`;
- `bridge_insecure` is false;
- event and state topics go out at QoS 1;
- command topics come in at QoS 0;
- the uplink bridge uses a persistent session;
- the downlink bridge uses a clean session.

Step 3: Verify queue storage and permissions

Run:

```
df -h /etc/mosquitto/data
mount
ls -ld /etc/mosquitto/data /etc/mosquitto/certs
ls -l /etc/mosquitto/data
ls -l /etc/mosquitto/certs
```

Pass this step only when:

- /etc/mosquitto/data is on persistent writable storage;
- the path is not under /tmp or another temporary filesystem;
- the mosquitto user can write the data directory;
- the private key is not world-readable;
- current free space is greater than the configured reserve.

The mosquitto.db file can be absent before Mosquitto has saved any persistent state. It should appear or change after the outage test creates queued messages.

Step 3A: Compare the gateway with the off-device security baseline

The buffer setup manual created <GATEWAY_EUI>-mqtt-security-baseline.txt on the administration workstation. This step independently recalculates the broker-configuration hash and certificate fingerprints and compares them with that approved off-device record.

On the gateway, create a fresh comparison file:

```
umask 077
{
  printf 'gateway_eui=%s\n' '<GATEWAY_EUI>'
  printf 'captured_utc=%s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  sha256sum /etc/mosquitto/mosquitto.conf /etc/mosquitto/conf.d/bridge.conf
  openssl x509 -in /etc/mosquitto/certs/<GATEWAY_EUI>.cert -noout -fingerprint -sha256 -serial -dates
  openssl x509 -in /etc/mosquitto/certs/ca.crt -noout -fingerprint -sha256 -subject -issuer
} > /tmp/gateway-mqtt-security-current.txt
cat /tmp/gateway-mqtt-security-current.txt
```

On the administration workstation, copy the current values:

```
scp root@<GATEWAY_IP>:/tmp/gateway-mqtt-security-current.txt \
./<GATEWAY_EUI>-mqtt-security-current.txt
```

Compare everything except the intentionally changing capture timestamp:

```
grep -v '^captured_utc=' ./<GATEWAY_EUI>-mqtt-security-baseline.txt \
> /tmp/gateway-mqtt-baseline.normalized

grep -v '^captured_utc=' ./<GATEWAY_EUI>-mqtt-security-current.txt \
> /tmp/gateway-mqtt-current.normalized

diff -u \
/tmp/gateway-mqtt-baseline.normalized \
/tmp/gateway-mqtt-current.normalized
```

Pass condition: diff prints nothing and returns exit status 0.

If it prints a difference, **stop gateway acceptance** and identify the exact changed configuration line or certificate. Do not overwrite the baseline just because the current gateway is online. For an intentional configuration change or certificate rotation, first complete the relevant setup procedure and all validation tests, document the approved change, then capture a new baseline from the known-good state.

Clean up temporary comparison files after review:

```
rm -f \  
  /tmp/gateway-mqtt-baseline.normalized \  
  /tmp/gateway-mqtt-current.normalized \  
  ./<GATEWAY_EUI>-mqtt-security-current.txt
```

On the gateway:

```
rm -f /tmp/gateway-mqtt-security-current.txt
```

This baseline detects later configuration/certificate drift. It does not prove that `mosquitto.db` message contents were never modified by a privileged attacker.

Step 4: Verify the loopback MQTT listener

Run:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

The listener must be:

```
127.0.0.1:1883
```

Fail this step when the listener is bound to `0.0.0.0`, `[::]`, a LAN address, or a WAN address. The local listener permits anonymous access only because it is restricted to loopback.

Step 5: Verify a local gateway event

Open an SSH session and subscribe to the local event topic:

```
mosquitto_sub -h 127.0.0.1 -p 1883 \  
  -t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/#' -v
```

Generate one uplink from the approved test device.

A message must appear under the expected region prefix and Gateway EUI. The payload uses Protobuf and may not be readable as text. The topic and message arrival are the important checks.

Press `Ctrl+C` after the message arrives.

When no local event appears, troubleshoot Concentrator, the device channel plan, MQTT Forwarder, and the loopback broker before checking the remote TLS bridge.

Step 5A: Verify one journal record for the same real uplink

Complete this step only after the reviewed journal implementation described in [Gateway 4A](#) has been built, pinned, and installed.

Generate one fresh real uplink while observing both the local MQTT event and the journal's documented status/inspection interface.

Record, without exposing unrestricted payloads in ordinary logs:

```
Gateway EUI
journal sequence
boot ID
PHYPayload or approved PHYPayload digest
record hash
previous-record hash
current segment ID
```

Pass when:

- exactly one new journal sequence corresponds to the known uplink;
- the Gateway EUI is correct;
- the journal source is the supported Concentrator event interface, not a copy taken after Node-RED or ChirpStack;
- recomputing the versioned record hash produces the stored hash;
- the record references the preceding valid hash;
- MQTT Forwarder still produces the normal local event independently.

If the journal implementation has no safe command/API for inspecting and verifying a staging record, **stop deployment and add one to the reviewed implementation**. Do not parse undocumented binary files with an improvised production script.

Step 6: Verify the remote broker connection

Empty broker container stdout does not prove that no MQTT traffic arrived. Verify the topic directly first:

```
sudo timeout 45 docker exec mosquitto mosquitto_sub \
-h 127.0.0.1 -p 1883 \
-u chirpstack -P <CHIRPSTACK_MQTT_PASSWORD> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats' \
-v -C 1
```

A binary-looking Protobuf payload is normal. If this succeeds while ChirpStack still shows Last seen: Never, the gateway bridge path is proven and troubleshooting moves to ChirpStack region/backend configuration and exact EUI registration.

Then inspect application stdout if needed:

```
docker compose logs --since=5m --tail=200 mosquitto chirpstack
```

Confirm that:

- gw-up-<GATEWAY_EUI> connects with the gateway certificate;
- gw-down-<GATEWAY_EUI> connects with the same gateway identity;

- the broker accepts event and state writes only for this Gateway EUI;
- the broker allows command reads only for this Gateway EUI;
- ChirpStack receives the regional gateway event topic.

Also confirm in the ChirpStack web interface that the registered gateway's **last seen** time updates after the test uplink.

Verify cross-gateway isolation

When a second test Gateway EUI is available, attempt to publish or subscribe outside this gateway's allowed topic path using the first gateway certificate. The remote broker must deny the request.

Do not weaken the ACL when this test fails. Correct the certificate identity, topic path, or per-gateway ACL rule.

Step 7: Verify normal LoRaWAN traffic

Use the approved OTAA test device.

1. Power-cycle or reset the device so it performs a fresh join when appropriate.
2. Confirm that OTAA succeeds.
3. Send one uplink with a known payload or device frame counter.
4. Confirm that ChirpStack shows the expected Gateway EUI for the received uplink.
5. Confirm that the application receives one canonical record for the uplink.
6. Schedule one harmless Class A downlink.
7. Confirm that the device receives the downlink in the expected receive window.

QoS 1 can redeliver an MQTT message after a reconnect. The application or database must prevent one radio uplink from creating duplicate canonical records.

Step 8: Verify WAN-outage buffering

Disconnect only the gateway's WAN path. Keep the Raspberry Pi, RAK5146, Concentrator, MQTT Forwarder, and local Mosquitto running.

Before disconnecting WAN, run:

```
stat -c 'size=%s modified=%y' /etc/mosquitto/data/mosquitto.db 2>/dev/null || \
ls -l /etc/mosquitto/data
df -h /etc/mosquitto/data
```

Then perform the test:

1. Disconnect the gateway WAN or block only the route to the remote broker.
2. Keep an SSH session through a separate management path when possible.
3. Generate a known number of real uplinks.
4. Confirm that the local `mosquitto_sub` test still receives the gateway event topics.
5. Confirm that ChirpStack does not receive the outage uplinks yet.
6. Check `/etc/mosquitto/data/mosquitto.db` again. It should be created or increase after Mosquitto saves the queued state.
7. Confirm that free space remains above `<BUFFER_FREE_SPACE_RESERVE_BYTES>`.

8. Restore WAN connectivity.
9. Confirm that the uplink bridge reconnects.
10. Confirm that all expected uplinks reach ChirpStack and the application.
11. Confirm that the local queue stops growing after it drains.

The delivered MQTT count can include QoS 1 redelivery. Validate the application result using unique device identifiers and frame counters rather than assuming every MQTT delivery is unique.

Continuous queue growth after WAN recovery means the remote DNS, route, TLS, certificate, broker ACL, topic prefix, or ChirpStack subscription is still failing.

Step 8A: Verify the journal during WAN outage and recovery

Run this together with Step 8 so the same real test uplinks exercise both paths.

Before WAN loss, capture the latest accepted server checkpoint:

```
checkpoint gateway ID
checkpoint sequence
checkpoint record hash
checkpoint segment hash
server receipt ID/time
```

Then disconnect only WAN/backhaul and generate the known test uplinks.

During the outage pass only when:

- journal sequence continues increasing;
- the current segment continues to append/rotate within its configured budget;
- closed unuploaded segments remain local;
- the cloud checkpoint does **not** advance while the gateway cannot reach the evidence endpoint;
- Mosquitto and the journal use separate storage budgets and the OS free-space reserve remains safe.

After WAN returns, verify through the server-side procedures in [Gateway Integrity](#):

1. missing closed segments upload;
2. their first chain link extends the last accepted pre-outage anchor;
3. record and segment hashes verify;
4. the independently captured remote MQTT gateway events match the journal records;
5. a new checkpoint is accepted only after continuity is established;
6. unresolved missing evidence is reported as `evidence_gap` and contradictory evidence as `integrity_failure`.

Do not mark outage telemetry gateway-verified merely because Mosquitto successfully drained it.

Step 9: Verify queue persistence across reboot

Repeat this test while the WAN remains unavailable:

1. Generate several uplinks and confirm that the local queue grows.
2. Reboot the gateway cleanly:

```
sh reboot
```

3. Wait for Gateway OS to return.
4. Reconnect over SSH.
5. Confirm that Concentrator, MQTT Forwarder, and Mosquitto restart.
6. Confirm that `/etc/mosquitto/data/mosquitto.db` still exists and contains the queued state.
7. Generate another uplink while WAN is still unavailable.
8. Restore WAN connectivity.
9. Confirm that both the pre-reboot and post-reboot uplinks reach ChirpStack.
10. Confirm that the application retains one canonical record per real uplink.

The buffer is not persistent when messages queued before reboot disappear.

Step 9A: Verify journal continuity across reboot

Perform this during the same controlled WAN-outage reboot test when possible.

Before reboot record:

```
last journal sequence
last record hash
boot ID
open/last closed segment
```

After reboot:

1. verify the journal recovers the last valid complete record;
2. verify the boot ID changes;
3. verify the next sequence continues instead of resetting;
4. generate a fresh uplink;
5. verify its `previous_record_hash` references the last valid pre-reboot chain state;
6. restore WAN and prove the server accepts continuity across the reboot boundary.

Only an incomplete final/torn record may be discarded automatically. A previously complete record with an invalid hash is an integrity incident, not normal crash cleanup.

Step 10: Verify that stale downlinks are not replayed

The downlink bridge uses QoS 0 and a clean session so old commands are not intentionally retained.

1. Disconnect the gateway WAN.
2. Attempt one harmless test downlink through ChirpStack.
3. Allow the downlink and its receive window to expire.
4. Restore WAN connectivity.
5. Confirm that the expired downlink is not transmitted after recovery.
6. Send a new Class A downlink after the next uplink.
7. Confirm that the new downlink succeeds.

Stop and correct the downlink bridge when an expired command is transmitted after reconnecting.

Step 11: Verify queue limits without filling the SD card

Perform this only in staging.

1. Confirm the configured `max_queued_messages` and `max_queued_bytes` values.
2. Confirm the current free-space reserve.
3. Disconnect WAN.
4. Generate controlled test traffic until the configured queue limit is approached.
5. Confirm that storage use remains bounded.
6. Confirm that the filesystem does not become full or read-only.
7. Restore WAN and verify that retained messages drain.
8. Identify which messages were retained or dropped using device frame counters.

The correct behavior is bounded loss at the configured limit while preserving the gateway filesystem. An unlimited queue or a full SD card is a failed result.

Step 12: Verify backup and restore

Use a spare microSD card so the working gateway card remains untouched.

1. Flash the same approved Gateway OS Base factory image.
2. Restore the latest protected LuCI backup.
3. Confirm that the Mosquitto configuration and certificate files are present.
4. Confirm the private-key permissions.
5. Boot the restored gateway with the RAK5146 installed.
6. Confirm that the Gateway EUI and channel plan match the original gateway.
7. Confirm that both MQTT bridges authenticate.
8. Repeat one local uplink, one remote uplink, one safe downlink, and one short WAN-outage test.

A backup is not proven until it has restored a working gateway on separate media.

Troubleshooting by failing layer

Concentrator has no stable Gateway EUI

Return to [03-configure-concentrator.md](#). Check the RAK5146 profile, SPI assembly, power, and exact channel plan.

Local MQTT receives no gateway event

Check MQTT Forwarder's backend, loopback address, QoS, topic prefix, client ID, and packet filters. Do not change the remote broker yet.

Local events work but the remote broker receives nothing

Check gateway DNS, UTC time, bridge certificate files, certificate/key match, broker hostname, and remote ACL.

The remote broker receives events but ChirpStack does not update

Compare `<REGION_TOPIC_PREFIX>` with the regions enabled on the ChirpStack server. Confirm the exact Gateway EUI registration and the broker topic subscription.

The queue does not grow during an outage

Confirm that MQTT Forwarder publishes at QoS 1, the uplink bridge has `cleansession false`, persistence is enabled, and the persistence directory is writable by `mosquitto`.

The queue grows during normal connectivity

The bridge is not successfully delivering. Check route, DNS, time, TLS, certificate identity, broker ACL, and topic prefix in that order.

Queued messages disappear after reboot

Confirm that `/etc/mosquitto/data` is persistent, Mosquitto can write it, and the service stops cleanly during a normal reboot. Recheck the persistence settings and autosave interval.

An old downlink is transmitted after reconnecting

Confirm that the downlink bridge uses `cleansession true`, the command topic is `in 0`, and no second MQTT or UDP path is retaining commands.

Final acceptance

The gateway is ready for normal operation when all of these are true:

- Gateway OS Base boots with correct time and network access.
- Concentrator initializes the RAK5146 with the approved channel plan.
- The Gateway EUI remains stable.
- MQTT Forwarder publishes Protobuf messages to `127.0.0.1:1883` at QoS 1.
- Local Mosquitto listens only on loopback.
- Both remote bridges authenticate with the gateway certificate.
- The remote ACL restricts access to this Gateway EUI.
- The current broker configuration and certificate fingerprints match the approved off-device baseline.
- A real OTAA join, uplink, and safe Class A downlink succeed.
- Uplinks remain buffered during WAN loss.
- The queue survives a clean gateway reboot.
- Buffered uplinks drain after WAN recovery.
- Duplicate MQTT delivery does not create duplicate canonical application records.
- Expired downlinks are not replayed.
- Queue limits protect the SD card.
- A protected backup restores successfully to spare media.
- UDP Forwarder remains disabled.
- The journal independently records the supported Concentrator source path and record/segment hashes verify.
- Journal sequence/hash continuity survives a clean reboot.
- WAN loss preserves both the Mosquitto queue and local journal within separate finite budgets.

- Recovery uploads missing journal segments and extends the last accepted cloud checkpoint.
- Server reconciliation can distinguish verified, evidence_gap, and integrity_failure.
- Operators understand that the cloud-anchored history is stronger than a local checksum, but the software-only unanchored offline tail is not claimed to be unforgeable against full gateway root compromise.

Continue with [Gateway operations](#) for registration, routine tests, backup, recovery, and troubleshooting.

Current flash-image baseline - 2026-09-01

Use this exact factory artifact for the next clean flash:

```
file=chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
bytes=28900364
sha256=bafe8b97baf9353df2654b1c8b71fa53d2ff764cd264d0ed6c924dd25a5ec67d
recovery=C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-as923-journal-20260901
```

Pre-flash acceptance already proved generated checksums, gzip/partition integrity, independent SquashFS extraction, RAK5146/SX1302 + AS923/as923 defaults, AS923-1 923.2/923.4 MHz channel files, SIM7600/QMI modules and uqmi, local Mosquitto, Concentrator/MQTT Forwarder, writer boot enablement, uploader default disablement, and no embedded gateway-evidence TLS secrets.

After flashing, validate only the hardware/runtime boundary: boot and writable overlay; RAK5146 Concentrator on AS923; SIM7600 tty/QMI enumeration; Mosquitto on 127.0.0.1:1883 with writable /etc/mosquitto/data; normal MQTT Forwarder enabled while UDP/mesh paths remain disabled; evidence writer running; uploader still disabled until mTLS provisioning; and one real uplink producing a journal record and reaching the telemetry path after protected bridge credentials are restored/provisioned.

Production MQTT bridge private keys/certificates and evidence-uploader mTLS material are intentionally not firmware contents. Keep them in the protected provisioning/recovery path.

Clean-overlay physical acceptance - 2026-09-02

Production Gateway-01 has now passed the post-flash hardware/runtime boundary after deliberately clearing stale writable OverlayFS state on the reused microSD. Independent extraction reconfirmed that the accepted SquashFS itself did not contain the stale MQTT/Evidence identities or commissioning /etc/hosts overrides. The gateway was reprovisioned from protected off-device credentials, rebooted, and reverified with:

Gateway EUI	0016c001f139a1cb
SX1302 model	rak_5146
region / channel plan	AS923 / as923
MQTT Forwarder	enabled, tcp://127.0.0.1:1883, qos=1
maintenance Ethernet	192.168.20.11/24, no default route
normal backhaul	Wi-Fi DHCP, default route via Wi-Fi
public MQTT	two established mTLS bridge sockets
Evidence	writer + uploader running, mTLS /readyz ready
commissioning host override	absent
ChirpStack last_seen_at	2026-09-02 01:30:01.955894+00 after reboot

Use ../automation/commission-clean-overlay.sh preflight before provisioning a future newly flashed/reused card. If production keys/bridge state appear before provisioning, stop and investigate/reset stale writable overlay rather than assuming the immutable image contains secrets.

The accepted 2026-09-01 SquashFS was also found to contain MQTT Forwarder QoS 0. Runtime commissioning corrected Gateway-01 to QoS 1 and the tracked image overlay is corrected for a future rebuild. The current gateway does not require a rebuild because its clean persisted runtime has already passed reboot and cloud verification.

Real post-reset assembled EMU-01 LoRaWAN traffic has created and uploaded gateway-evidence segments/checkpoints successfully. The gateway writer/uploader, public mTLS ingest path, checkpoint/segment receipts, and cloud trusted-decoder verification are physically proven. At the 2026-09-02 powered-off acceptance boundary, the last EMU-01 application uplink was frame counter 666 at 05:31:45.795 UTC; Gateway-01 subsequently closed and receipted segment 28, journal/open was empty, and the cloud verifier reported all 492 selected v2 events verified with no evidence failures. This closes the Gateway OS physical evidence-lineage boundary; later sensor work should re-check it only if the gateway/evidence configuration changes.

Supporting runbook: 06-rf-planning-and-site-survey.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

RF Planning, Antennas, and Site Survey

Use this guide to estimate link margin, select an antenna and cable, check Fresnel clearance, identify nearby interference, and document the installation site. Calculations are planning inputs, not guaranteed coverage.

1. Link budget and propagation

The reliability of a LoRaWAN gateway deployment depends on maintaining a positive **Link Margin**. The link margin determines whether a field node's transmitted signal can overcome path loss and arrive at the gateway receiver above its sensitivity threshold.

1.1 Received Power Equation (Friis Transmission Equation)

$$P_{RX} = P_{TX} + G_{TX} - L_{TX} - FSPL + G_{RX} - L_{RX}$$

Where: - P_{RX} : Received Signal Power at Gateway Concentrator (dBm) - P_{TX} : Field Node Transmit Power (typically +14 dBm to +22 dBm) - G_{TX} : Transmitter Antenna Gain (dBi) - L_{TX} : Transmitter Coaxial Cable & Connector Loss (dB) - $FSPL$: Free Space Path Loss (dB) - G_{RX} : Gateway Antenna Gain (dBi) - L_{RX} : Gateway Coaxial Cable & Connector Loss (dB)

1.2 Free Space Path Loss (FSPL) Formula

$$FSPL (dB) = 20 \log_{10}(d) + 20 \log_{10}(f) + 32.44$$

Where: - d : Distance between node and gateway in kilometers (km) - f : RF Operating Frequency in megahertz (MHz)

Example: 923 MHz over 5 km in free space

$$FSPL = 20 \log_{10}(5) + 20 \log_{10}(923) + 32.44 \quad FSPL = 13.98 + 59.30 + 32.44 = 105.72 \text{ dB}$$

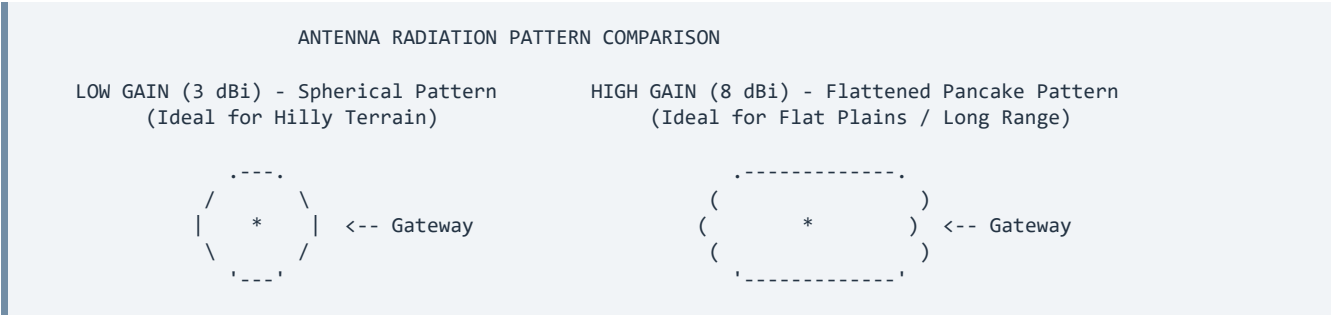
1.3 Receiver Sensitivity Formula

The minimum receivable RF signal level (S) for an SX1302/SX1303 baseband receiver is calculated as:

$$S = -174 + 10 \log_{10}(BW) + NF + SNR_{\text{min}}$$

For LoRa SF12 at 125 kHz Bandwidth ($BW = 125000 \text{ Hz}$), with Noise Figure $NF = 6 \text{ dB}$ and $SNR_{\text{min}} = -20 \text{ dB}$: $S = -174 + 50.96 + 6 + (-20) = -137.04 \text{ dBm}$

2. Antenna gain, pattern, polarization, and VSWR



2.1 Antenna Gain (dBi vs dBd)

- dBi : Decibels relative to an theoretical isotropic radiator (radiates equally in all 3D directions).
- dBd : Decibels relative to a half-wave dipole antenna.
- Conversion Formula:** $\text{dBi} = \text{dBd} + 2.15$

2.2 Gain vs Terrain Matching Matrix

- 3 dBi - 5 dBi Omni Antenna:** Best for hilly, mountainous, or urban environments with high elevation variation. Energy is radiated in a broad spherical beam, capturing signals from both valleys and elevated ground.
- 8 dBi - 12 dBi Omni Antenna:** Best for flat rural plains, agricultural fields, or coastal monitoring. Compresses RF energy into a tight horizontal pancake pattern, maximizing long-range horizon reach.

[!CAUTION] **Over-Gain Warning:** Installing a high-gain (12 dBi) antenna on a high tower in a hilly area creates a "blind spot" directly beneath the tower. Field nodes located close to the base of the tower will fall outside the vertical beam pattern.

2.3 Voltage Standing Wave Ratio (VSWR)

VSWR measures how effectively RF power is transferred from the concentrator module into the antenna without reflecting back.

VSWR Value	Return Loss (dB)	Reflected Power (%)	Operational Status
1.0 : 1	∞	0 %	Ideal reference
1.2 : 1	20.8 dB	0.8 %	Very good
1.5 : 1	14.0 dB	4.0 %	Common acceptance target
2.0 : 1	9.5 dB	11.3 %	Investigate cable, connectors, and antenna
> 3.0 : 1	< 6.0 dB	> 25.0 %	Stop transmit testing and correct the RF path

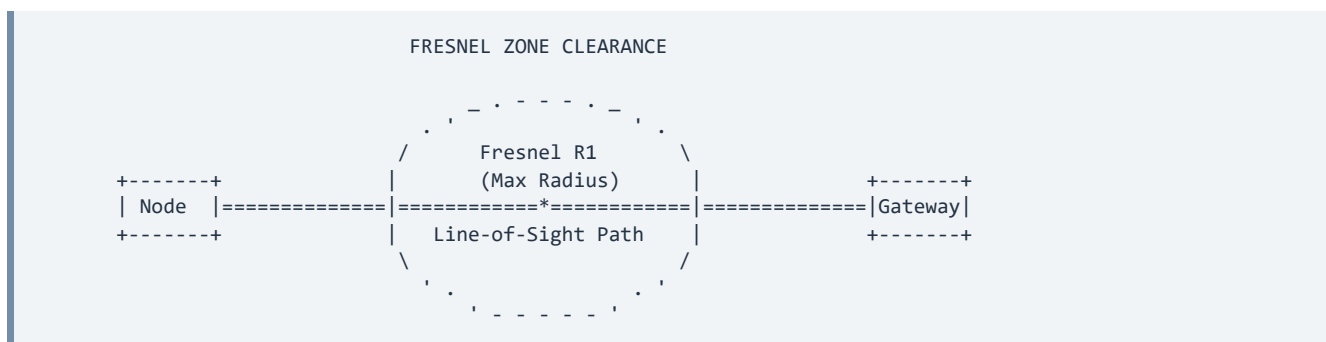
3. Coaxial-cable loss near 900 MHz

Coaxial cable quality directly impacts gateway performance. High cable losses waste transmit power and degrade receiver sensitivity.

Coaxial Cable Model	Outer Diameter	Attenuation per 10m @ 900 MHz	Attenuation per 30m @ 900 MHz	Flexibility & Usage
RG-174	2.8 mm	9.8 dB loss	29.4 dB loss	Internal pigtails only (< 30 cm)
RG-58	5.0 mm	4.5 dB loss	13.5 dB loss	Short bench test jumpers (< 2 m)
LMR-200	4.9 mm	2.9 dB loss	8.7 dB loss	Short mast runs (< 5 m)
LMR-400	10.3 mm	1.28 dB loss	3.84 dB loss	Primary Outdoor Tower Cable Standard
LMR-600	15.0 mm	0.86 dB loss	2.58 dB loss	Long commercial tower runs (> 20 m)

4. Fresnel-zone clearance

The 1st Fresnel Zone is an elliptical region surrounding the direct line-of-sight path between node and gateway. If obstacles (trees, buildings, ground terrain) intrude into this zone by more than 20%, severe RF diffraction fading occurs.



4.1 Fresnel Radius Formula at Midpoint

$$R_1 = 8.65 \sqrt{\frac{d_1 \cdot d_2}{f \cdot (d_1 + d_2)}}$$

Where: - R_1 : First Fresnel zone radius in meters (m) - d_1, d_2 : Distances from obstacle to node and gateway in kilometers (km) - f : Frequency in gigahertz (e.g., 0.923 GHz)

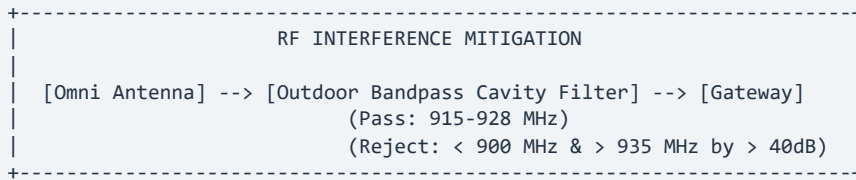
Calculation Example (10 km Total Distance, Obstacle at 5 km Midpoint):

$$R_1 = 8.65 \sqrt{\frac{5 \cdot 5}{0.923 \cdot (5 + 5)}} = 8.65 \sqrt{\frac{25}{9.23}} = 8.65 \sqrt{2.708} = 8.65 \cdot 1.645 = 14.23 \text{ meters}$$

Planning target: keep most of the first Fresnel zone clear. The exact acceptance margin depends on terrain, reliability target, foliage, and measured field performance.

5. Nearby transmitters and filtering

When a RAK gateway is installed on a commercial cell tower, nearby **4G/5G LTE transmitters** (especially LTE Band 8 / Band 5 operating in 850-900 MHz) can overload the SX1250 RF front-end transceivers, causing receiver desensitization ("desense") and packet drops.



- **Solution:** Install an inline **High-Q SAW or Cavity Bandpass Filter** tuned specifically to your regional sub-band (e.g., passband 915-928 MHz for US915/AS923, passband 863-870 MHz for EU868) between the outdoor antenna and the gateway.

6. Perform the site survey

Step 1: Describe the radio path

At the proposed gateway location, note antenna height, mounting point, terrain, nearby structures, vegetation, and expected device locations. These observations explain later dead zones and help choose antenna gain and height.

Healthy evidence is a plausible line-of-sight or diffraction path to the service area. Major metal obstructions, changing container stacks, or heavy foliage mean coverage must be proven with repeated field tests rather than a desktop estimate.

Step 2: Verify power and backhaul

Test the actual power source, UPS runtime when required, and the real Ethernet or 4G path to the MQTT endpoint. Measure normal latency, packet loss, DNS behavior, and a controlled outage/recovery.

The backhaul is healthy when it remains stable during radio activity and the local MQTT queue drains after recovery. Reconnect loops, power warnings, or a queue that never drains must be fixed before permanent mounting.

Step 3: Measure the antenna system

With suitable RF equipment, measure the complete antenna, connector, pigtail, and cable path in the approved operating band. Compare VSWR and insertion loss with the antenna and cable requirements.

Stop transmit testing when the result indicates a damaged cable, wrong-band antenna, poor connector, water ingress, or excessive reflected power. Correct the RF path before compensating with higher transmit power.

Step 4: Check interference and installation safety

Survey the approved band for persistent interference and strong nearby transmitters. Have a qualified installer verify grounding, lightning/surge protection, cable entry, weather sealing, mast loading, and local electrical requirements.

Step 5: Run representative field uplinks

Send repeated uplinks from representative near, far, obstructed, and edge-of-coverage locations. Compare RSSI, SNR, data rate, retries, received-versus-expected messages, and behavior across time and site conditions.

A successful survey demonstrates adequate margin and repeatability for the required reporting interval. A single received packet is not coverage evidence; abnormal loss or unstable link metrics require placement, antenna, filtering, or gateway-count changes followed by another test.

Canonical source: [deployment/technology-guides/02-rak5146-concentrator-d-as923.md](#). For any mutable production-status statement, check the dated baseline and verified live state.